Experiment-1

Aim:- Study of PROLOG.

PROLOG stands for Programming in Logic — an idea that emerged in the early 1970's to use logic as programming language. It is a programming language that's well-suited for developing logic-based artificial intelligence applications. The early developers of this idea included Robert Kowaiski at Edinburgh (on the theoretical side), Marrten van Emden at Edinburgh (experimental demonstration) an Alian Colmerauer at Marseilles (implementation). David D.H. Warren's ef cient implementation at Edinburgh in the mid - 1970's greatly contributed to the popularity of PROLOG.

It is a declarative programming language designed for developing logic-based AI applications. Developers can set rules and facts around a problem, and then Prolog's interpreter will use that information to automatically infer solutions.

It is a programming language centred around a small set of basic mechanisms, including pattern matching, tree-based data structuring and automatic backtracking. This Small set constitutes a surprisingly powerful and flexible programming framework. PROLOG is especially well suited for problems that involve objects-in particular, structured objects-and relations between them.

Facts, Rules, Queries, Predicates and Clauses:

A Prolog program consists of a number of clauses. Each clause is either a fact or a rule. After a Prolog program is loaded {or consulted) in a Prolog interpreter, users can submit goals or queries, and the Prolog intepreter will give results (answers) according to the facts and rules.

Facts: A fact must start with a predicate (which is an atom) and end with a full stop. The predicate may be followed by one or more arguments which are enclosed by parentheses. The arguments can be atoms (in this case, these atoms are treated as constants), numbers, variables or lists. Arguments are separated by commas. For example: mantjohn).

If we consider the arguments in a fact to be objects, then the predicate of the fact describes a property of the objects. In a Prolog program, a presence of a fact indicates a statement that is true. An absence of a fact indicates a statement that is not true.

Rules: A rule can be viewed as an extension of a fact with added conditions that also have to be satisfied for it to be true. It consists of two parts. The first part is similar to a fact (a predicate with arguments). The second part consists of other clauses (facts or rules which are separated by commas) which must all be true for the rule itself to

be true. These two parts are separated by ":-". You may interpret this operator as "if" in English. not(X,Y) :- man(X), woman(Y).

Queries: A query is a statement starting with a predicate and followed by its arguments, some of which are variables. Similar to goals, the predicate of a valid query must have appeared in at least one fact or rule in the consulted program, and the number of arguments in the query must be the same as that appears in the consulted program.

The purpose of submitting a query is to find values to substitute into the variables in the query such that the query is satisfied. This is similar to asking a question, asking for "what values will make my statement true". For example: notgohn, mary)?

Predicates:

• A predicate is a function or relation that takes some arguments and returns a boolean value (true or false). Predicates represent properties or relations in a logical system.

• Example: In the predicate cat(mittens), "cat" is the predicate, and "mittens" is the argument.

Clauses:

• A clause in logic typically refers to a rule or a fact. In Prolog, a clause can be a fact or a rule. A clause is composed of a head and a body.

1. Fact: cat(mittens).
2. Rule: mammal(X) :- cat(X).

• Clauses are used to define relationships between different facts in a knowledge base.

Features of Prolog:

- Declarative Nature: Focuses on *what* needs to be solved rather than *how* to solve it.
- Logical Syntax: Uses facts, rules, and queries to represent relationships and logic.
- Backtracking: Automatically searches for all possible solutions by backtracking when it hits a dead-end.
- Unification: Matches terms in queries with facts and rules by finding substitutions for variables.
- Pattern Matching: Compares and matches terms in the query with those in the knowledge base.
- Non-determinism: Can return multiple solutions to a problem by exploring different possible answers.

- Recursion: Commonly used to process data and define relationships, especially with lists or trees.
- Built-in Predicates**:** Provides pre-defined predicates for tasks like arithmetic, list manipulation, and more.
- Dynamic Database: Allows the addition and removal of facts and rules during program execution (asserUretract).
- Constraint Logic Programming (CLP): Extends Prolog for solving problems with constraints, such as optimization or scheduling.
- Natural Language Processing (NLP): Well-suited for parsing and interpreting languages, often used in AI and linguistics.
- Built-in Lists and Data Structures: Native support for lists and other common data structures for handling collections of terms.
- No Side Effects: Purely declarative, with no hidden state or side effects during computation.
- Interactivity: Supports interactive development where users define and query facts, rules, and conditions in real-time.

Symbolic Language:

PROLOG is a programming language for symbolic, non-numeric computation. It is especially well suited for solving problems that involve objects and relations between objects. For example, it is an easy exercise in PROLOG to express spatial relationship between objects, such as the blue sphere is behind the green one. It is also easy to state a more general rule: if object X is closer to the observer than object Y. and object Y is closer than Z, then X must be closer than Z. PROLOG can reason about the spatial relationships and their consistency with respect to the general rule. Features like this make PROLOG a powerful language for ArtJIcia1 LanguageA1,) and non- numerical programming. There are well-known examples of symbolic computation whose implementation in other standard languages took tens of pages of indigestible code, when the same algorithms were implemented in PROLOG, the result was a crystal-clear program easily fitting on one page.

1. It uses symbols to represent facts, rules, and queries in its knowledge base.
2. Symbols in Prolog include constants, variables, predicates, and operators.
3. Logical relationships are expressed through symbolic rules, using operators like :- (if) and , (and).
4. Prolog's symbolic nature makes it ideal for applications like artificial intelligence and knowledge representation.
5. The combination of symbols and logical inference allows Prolog to solve complex problems through reasoning.

SYMBOLS USED IN PROLOC:

Symbol	Name	Usage
`.`	Full stop	Used to terminate clauses or queries.
`:-`	Rule operator	Denotes "if" in rules, separating the head from the body.
`,`	Comma	Represents a logical AND in Prolog queries or rule bodies.
`;`	Semicolon	Represents a logical OR in queries or rules.
`=`	Equality operator	Used to test if two terms are the same.
`\=`	Inequality operator	Tests if two terms are not the same.
`==`	Strict equality operator	Tests if two terms are identical without variable bindings.
`\==`	Strict inequality	Tests if two terms are not identical without variable bindings.
`is`	Arithmetic evaluation	Evaluates an arithmetic expression and unifies it with a variable.
`=<`	Less than or equal to	Used for arithmetic comparisons.
`<`	Less than	Used for arithmetic comparisons.
`>=`	Greater than or equal to	Used for arithmetic comparisons.
`>`	Greater than	Used for arithmetic comparisons.

INSYALLATION STEPS:

Step 1: Visit *swi-prolog.org* website using any web browser.

Step 2: Click on Download which is adjacent to Home, dropdown list will appear then click on SWI-Prolog.

Step 3: New webpage will open, click on Stable release.

Step 4: After clicking on stable release new webpage will open which will contain stable versions of prolog for different platforms. Under binaries there are two stable releases for windows, first is SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit) and the other is SWI-Prolog 8.4.0-1 for Microsoft Windows (32 bit). Click on the one as per your system configuration. Let's take the one for 64-bit operating system.

Step S. After clicking on SWI-Prolog 8.4.0-1 for Microsoft Windows (64 bit), a new webpage will open, check on I understand checkbox to make the download link active. Then click on the download link, downloading of the executable Ie will start shortly. It is a small 11.9 MB file that will hardly take a minute.

Step 6: Now check for the executable file in downloads in your system and run it. Step 7:

It will prompt confirmation to make changes to your system. Click on Yes. Step 8: Setup

screen will appear, click on Next.

Step 9: The next screen will be of License Agreement, click on I Agree.

Step 10: After it there will be screen of installing options so check the box for Add swipl to the system path for all users, and also check the box for create a desktop icon and then click on the Next button.

Step 11: The next screen will be of installing location so choose the drive which will have sufficient memory space for installation. It needed only a memory space of 50 MB.

Step 12: Next screen will be of choosing Start menu folder so don't do anything just click on Next Button.

Step 13: This last screen is of choosing components, all components are already marked so don't change anything just click on Install button.

Step 14: After this installation process will start and will hardly take a minute to complete the installation.

Step 15: Clickon FinishaRertheinstallatonprocessiscomplete.

Step 16: SWI Prolog is successfully installed on the system and an icon is created on the desktop.

Step 17: Run the software and see the interface.

Experiment-2

Aim: Write simple fact for the statements using PROLOG

1) Ram likes mango.
2) Seema is a girl.
3) Bill likes Cindy.
4) Rose is red.
5) John owns gold.

CODE:

```
f Facts
likes (Ram, mango).
girl(seema).
likes (bill, cindy).
red (rose).
owns (john,gold).
```

OWPUT:

querying ”likes{Ram, mango)“

?- likes(Ram, What).
What - mango.

?- likes(Who, mango).
Who = Ram.

?- likes(Ram, mango).
true.

?- 1ikes(seema, mango).

false.

querying “gir1(seema)*

?- girl(Who). to-
seema.

?- gir1(seema).
true.

?- girl(Ram).
fa1se.

Querying ’likes(bill, cindy)’

?- likes(bill, what).
what = cindy.

?- likes(who, cindy). who - bill.

?- likes(bill, cindy).
true.

?- likes(Ram, cindy).
false.

Querying ‘red (rose)’

?- red(What).
What=rose.

?- red(Who).
Who = rose.

?- red(rose).
true.

?- red(mango) .

false.

Querying ’owns(john, gold)’

*- owns { john,What) .
what - gold.

?- owns(who, gold). who -
john.

?- owns(john, gold).
true.

?- owns(bill, gold). false.

?- owns(Who, What).
Who = john,
What = gold.

Experiment-3

Aim: Write predicates, one converts centigrade temperatures to Fahrenheit, the other checks if a temperature is below freezing using PROLOG

CODE:

```
c_tof(C,F):- F is C * 9/5 + 32.
freezing(F):- F =< 32.
```

OUTPUT:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)

SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run 7- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use 7- help(Topic). or 7- apropos(Word).

7- freezing(30).
true.

7- freezing(36).
7a1se.

?- freezing(20).
true.

7- c_tof(20,F).
F = 68.

7- c_tof(-2S,F).
F = -13.
```